Project Report On

Movie Ticket Booking System



COMPUTER SCIENCE

PROJECT FILE

SUBJECT CODE: 083



NAME:

CLASS:

ROLL NO:

SUJAY PUBLIC SCHOOL
MANAGED BY N. MUTHUSWAMY EDUCATIONAL TRUST
AFFILIATED TO CBSE (K-12) | AFFILIATION NO: 1931412 | SCHOOL CODE: 56439
Vadakkupattu Main Rd, Medavakkam, Chennai - 600 100.
(+91) 98402 55776 / (+91) 99405 66997 /sujaypublicschool
info@sujaypublicschool.com | https://sujaypublicschool.com/

CERTIFICATE

Certified that this is a Bonafide record of Project work done by

_____ Roll No. _____ of class _____ in the Laboratory during the Year _____

Signature of the Teacher Signature of the Principal

Submitted for the _____ _____ Examination, held on _____.

Date:

Internal Examiner External Examiner

Contents

1. Introduction

2. Objective and Scope of the Project

3. System Implementation

3.1 Hardware Used
3.2 Software Used
3.3 Product Functions

4. Theoretical Background

4.1 What is Python?
4.2 What is Database?
4.3 What is MySQL?

5. System Design and Development

5.1 Database Design
5.2 Flowchart
5.3 Flowchart Description
5.4 User Case Diagrams

6. Event Coding

7. Output

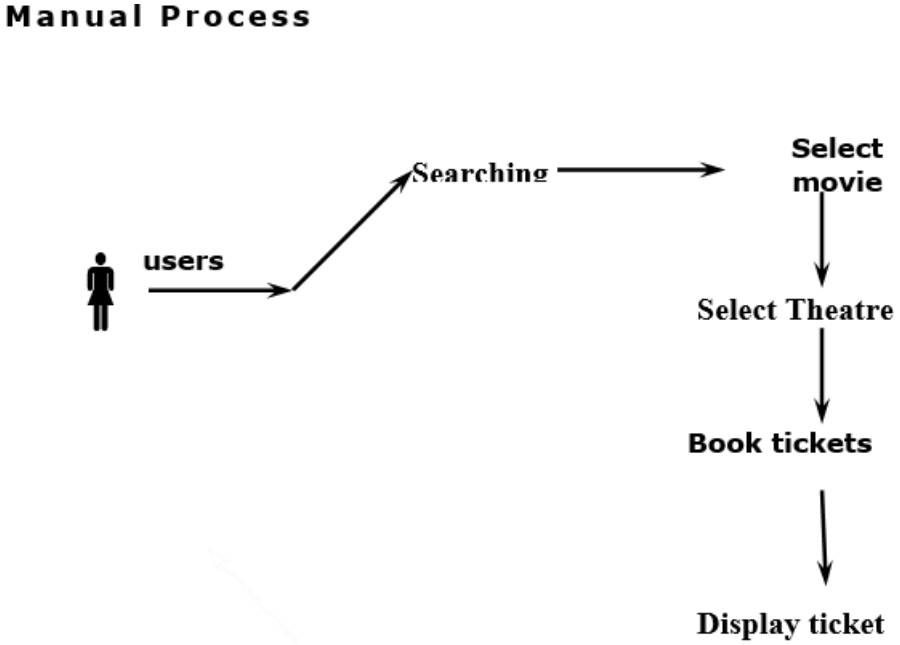
8. Conclusion

9. Bibliography

1. Introduction

Movie theatres have become one of the most preferred getaways in today's world. And yet going to the theatre, selecting a movie, and making sure the tickets aren't sold out, could prove tedious. Any knowledge about the list of movies, show timings, or even seats can be attained by the customer only by being present physically.

An **Online Movie Ticket Booking System** brings the theatres closer to the customers. It eases and improves the booking experience. The system holds the database of all the movies being played along with their show timings and available seats. This system can be accessed using a website by the customers a few days before the show to book their seats of choice, and at the same time, keeps the theatre admins updated about the seats being booked.



2. Objective and Scope of the Project

The main objective of our online ticket booking system is to provide another way for the user to buy cinema ticket.

It is an automatic system.

After inserting the data to database, staff need not to due with the order receive through the system. In fact, there is similar system on the internet, but there is no refund method found in the existing system.

This system is basically aimed to provide the user the complete information of the movie, according to which the user can book the tickets and along with can order snacks and the refund facility provides more flexibility to the system.

The goals of our system are:

1. To provide an anytime anyplace service for the user.
2. To minimize the number of staff at the ticket box.
3. To promote the film on the internet.
4. To increase the profit to obtain statistic information from the booking record.

3. System Implementation

3.1 Hardware Used

- OPERATING SYSTEM - WINDOWS 7 AND ABOVE
- PROCESSOR - PENTIUM(ANY) OR AMD ATHALON(3800+- 4200+ DUAL CORE)
- MOTHERBOARD - 1.845 OR 915,995 FOR PENTIUM 0R MSI
    - K9MM-V VIA K8M800+8237R PLUS CHIPSET FOR AMD ATHALON
- 4. RAM - 512MB+
- 5. Hard disk - SATA 40 GB OR ABOVE
- 6. CD/DVD r/w multi drive combo - (If back up required)
- 7. FLOPPY DRIVE 1.44 MB - (If Backup required)
- 8. MONITOR 14.1 or 15 - 21 inch
- 9. Key board and mouse
- 10. Printer - (if print is required – [Hard copy])

3.2 Software Used

- Windows OS
- Python
- MYSQL
- MYSQL python connector

3.3 Product Functions

i. Register

a. Input – Name , email-id , contact No. , Password
b. Processing – User will be registered
c. Output – User's account has been created successfully

ii. Login

a. Input – Id Name ,Password
b. Processing – credentials will be checked to verify the user or admin
c. Output – Go to Home page

iii. Book Ticket

a. Input – Movie name , date , time , seat type , No. of tickets
b. Processing – credentials of movie will be checked that desired seat is available or not .
c. Output – movie will be booked

iv. Order Snacks

a. Input – Food item ,No. of Item
b. Processing – snack will be check that it is available or not
c. Output – snack will be ordered

v. Payment

a. Input – Card No. , CVV No.
b. Processing – details are verified
c. Output – payment done successfully

vi. Send Notification

a. Input – none
b. Processing – sends conformational message

vii. Cancel Ticket

a. Input – user Id , Password , Ticket Id
b. Processing – cancellation process is start by checking that it fulfill cancellation constraint
c. Output – Ticket will be cancelled with/without refund .

4. Theoretical Background

4.1 What is Python?

Python is a high-level, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation. Python is dynamically-typed and garbage-collected. It supports multiple programming paradigms, including structured, object-oriented and functional programming.

4.2 What is File Handling?

File handling is an integral part of programming. File handling in Python is simplified with built-in methods, which include creating, opening, and closing files. While files are open, Python additionally allows performing various file operations, such as reading, writing, and appending information.

4.3 What is a Database?

A **database** is an organized collection of data stored and accessed electronically. Small databases can be stored on a file system, while large databases are hosted on computer clusters or cloud storage. The design of databases spans formal techniques and practical considerations, including data modeling, efficient data representation and storage, query languages, security and privacy of sensitive data, and distributed computing issues, including supporting concurrent access and fault tolerance.

4.4 What is MySQL?

MySQL is an open source relational database management system (RDBMS) based on Structured Query Language (SQL). MySQL runs on virtually all platforms, including Linux, UNIX and Windows. Although it can be used in a wide range of applications, MySQL is most often associated with web applications and online publishing.

5. System design and Development

5.1 Database Design

A Data Design is a collection of names, definitions, and attributes about data elements that are being used or captured in this software project. It describes the meanings and purposes of data elements within the context of this project, and provides guidance on interpretation, accepted meanings and representation so that user and analyst will have a common understanding of inputs, outputs and other project components.

1. **Name :Register**

S No.	Field Name	Data Type	Field Length	Constraints	Description	Example
1.	Name	Char	50	Not Null	To store nameofthe customer	Abc
2.	Email Id	Varchar	30	Not Null, primary key	Login id for the customer	abc@gmail.com
3.	Password	Varchar	10	Not Null	Login password of user, must contain at least 8 characters including special ones	abc@123
4.	Phone No.	Integer	10	Not Null, unique	Must be 10 digits long	9812467223

2. **Name :Admin**

SNo.	Field Name	Data Type	Field Length	Constraints	Description	Example
1.	Adminid	Varchar	30	Not Null, primarykey	Any valid email id	adminMovieBasket@gmail.com
2.	Password	Varchar	10	Not Null	Login password for admin, must contain at least a number or special character	Admin@123

3. **Name : Login**

S No.	Field Name	Data Type	Field Length	Constraints	Description	Example
1.	LoginId	Varchar	30	Not Null,primary key	Login id of the user	abc@gmail.com
2.	Password	Varchar	10	Not Null	Login password of the user, must contain at least a number or a specialcharacter	abc@123

4. **Name : Movie**

S No.	Field Name	Data Type	Field Length	Constraints	Description	Example
1.	Movie name	Char	30	Primary key, not null	Name of movie	Simba , Kalank
2.	Date	Date	10	Not null	yyyy-mm-dd	2019-04-17
3.	Time	Varchar	10	Not null	Specifies movie screening time	12 pm
3.	Seat Type	Char	10	Not null	Different types of seats	Silver, gold, platinum
4.	Number of Tickets	Integer	20	Not null	Specifies quantity	2

5. **Name : Snacks**

S No.	Field Name	Data Type	Field Length	Constraints	Description	Example
1.	Food Item	Char	10	Not null	Specifies type of food	Popcorn
2.	Number of Item	Integer	10	Not null	Specifies its quantity	2

6. **Name : Payment**

S No.	Field Name	Data Type	Field Length	Constraints	Description	Example
1	Pid	Varchar	20	Primary key, not null	Unique id for particular payment	1234
2	pamt	Integer	10	Not null	Total amount	500
3.	P_date	Date	10	yyyy-mm-dd	Date of payment	21/3/2019
4.	P_cid	Varchar	20	Not null	Payment for a specific customer	233

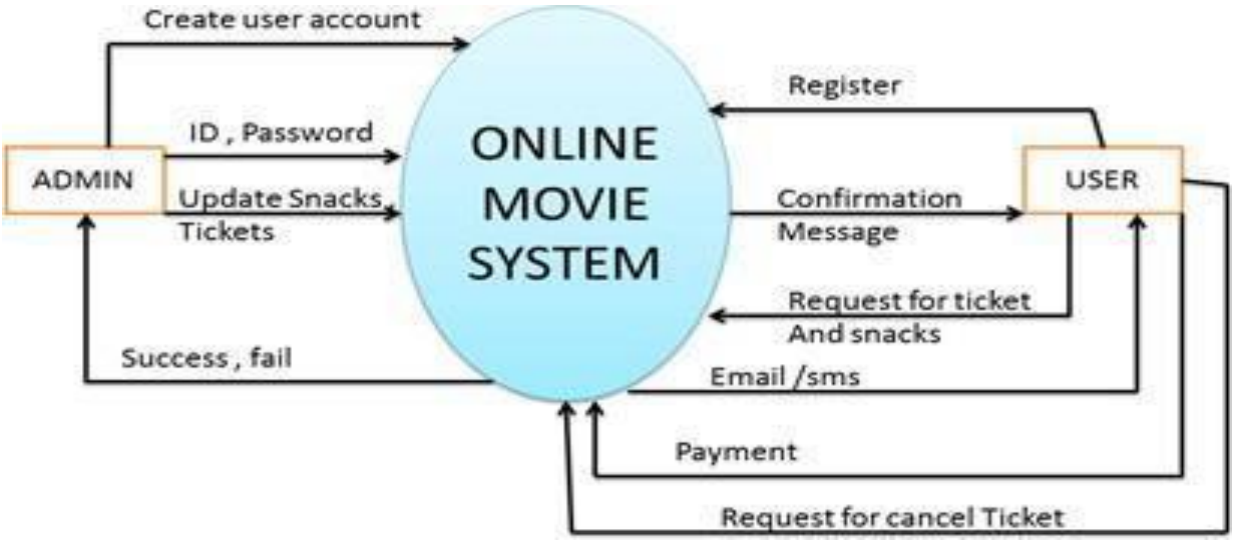
7. **Name : Cancel Ticket**

S No.	Field Name	Data Type	Field Length	Constraints	Description	Example
1.	Userid	Varchar	10	Primary key , not null	User id	1234
2	Pid	Varchar	10	Primary key , not null	Payment id	245
3.	Refund	Char	10	null	Refund amount	300

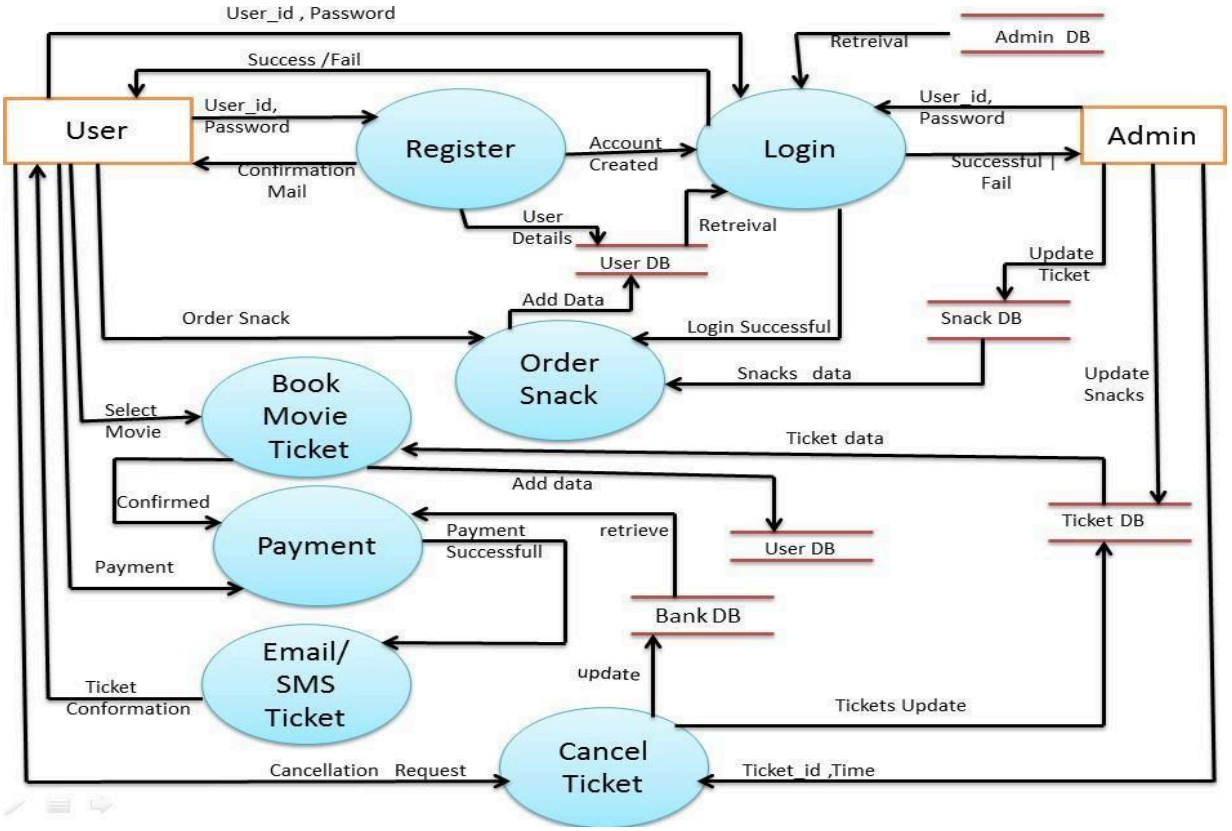
8. **Name: Discount**

S No.	Field Name	Data Type	Field Length	Constraints	Description	Example
1.	Price	Integer	10	Not null	amount	50
2.	CouponId	varchar	10	Primary , not null	Coupon id for a specific discount	3454
3.	M_name	varchar	15	Not null	Movie name	Abc

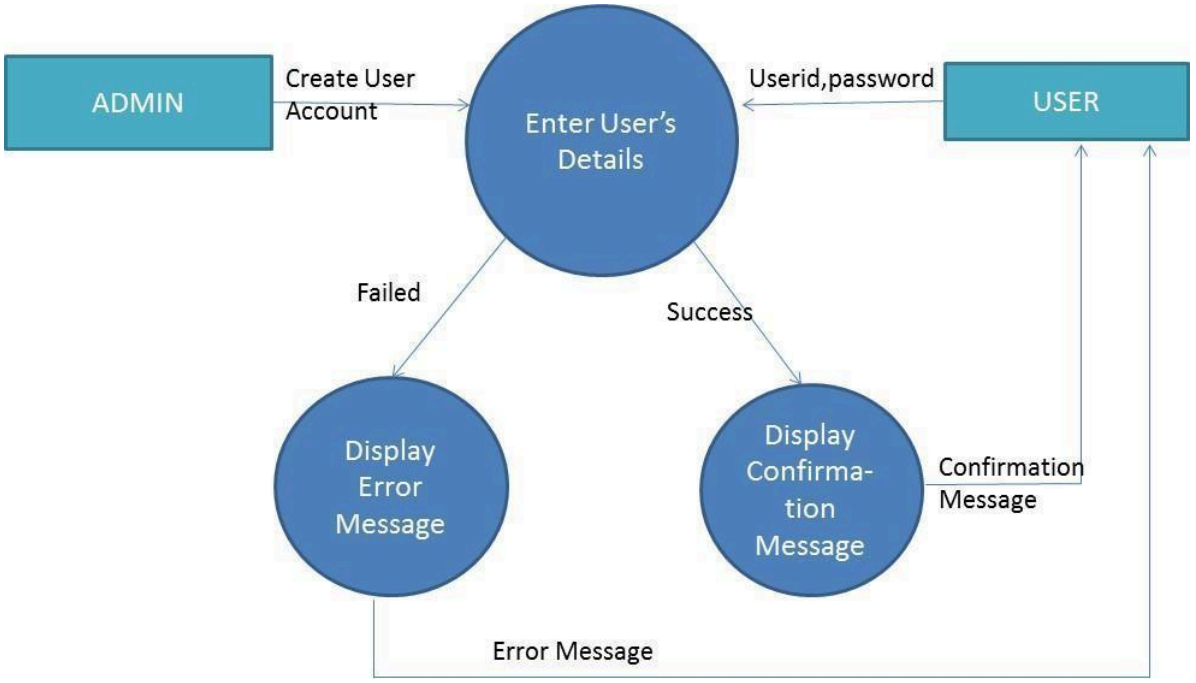
5.2 Flowchart



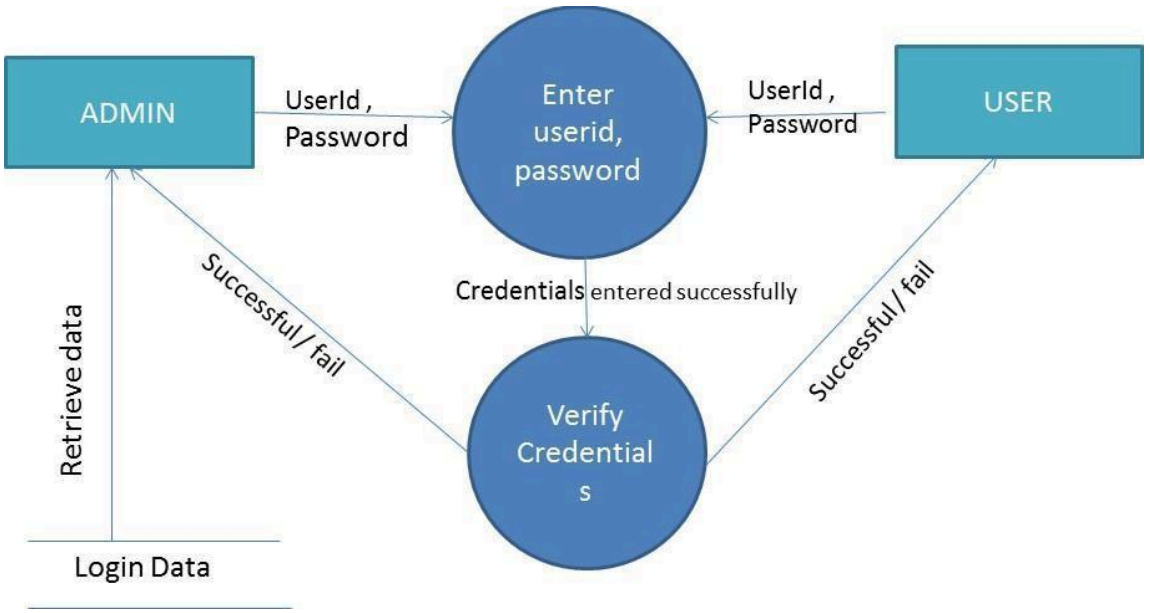
Context:



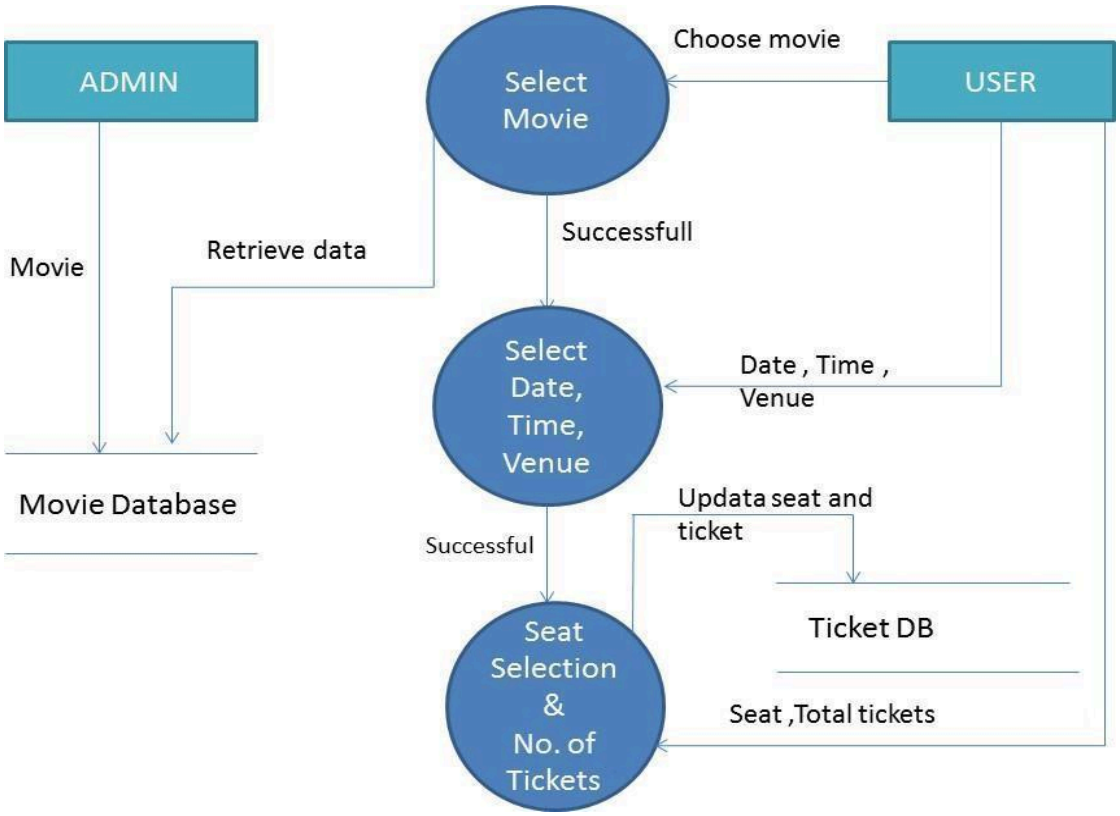
Register Module:



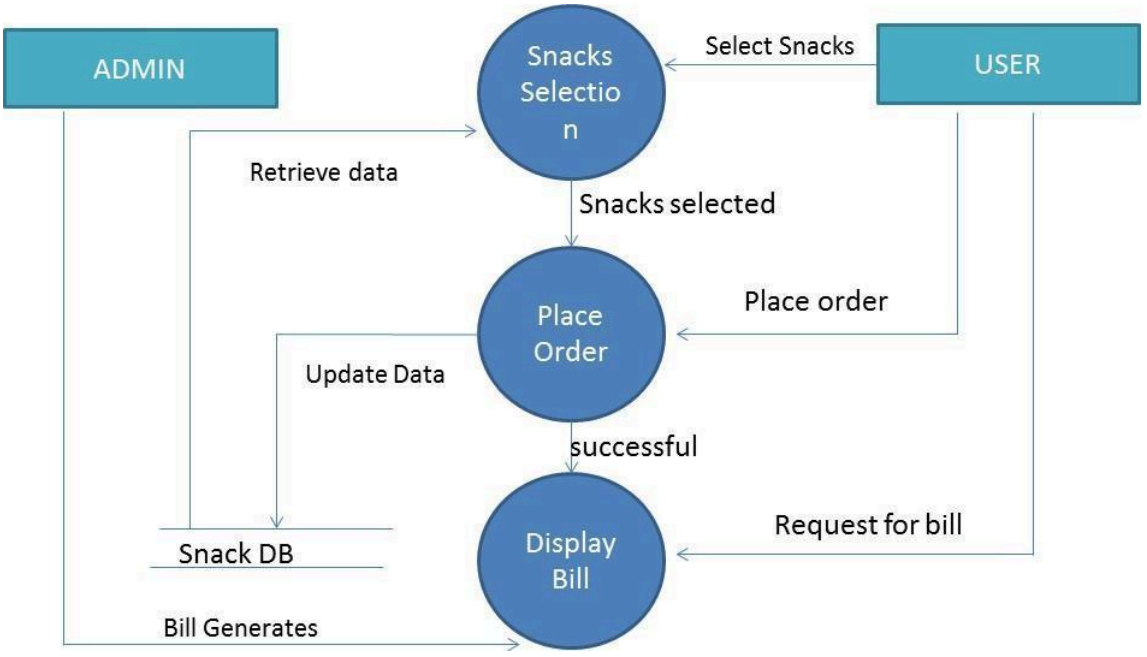
Login Module:



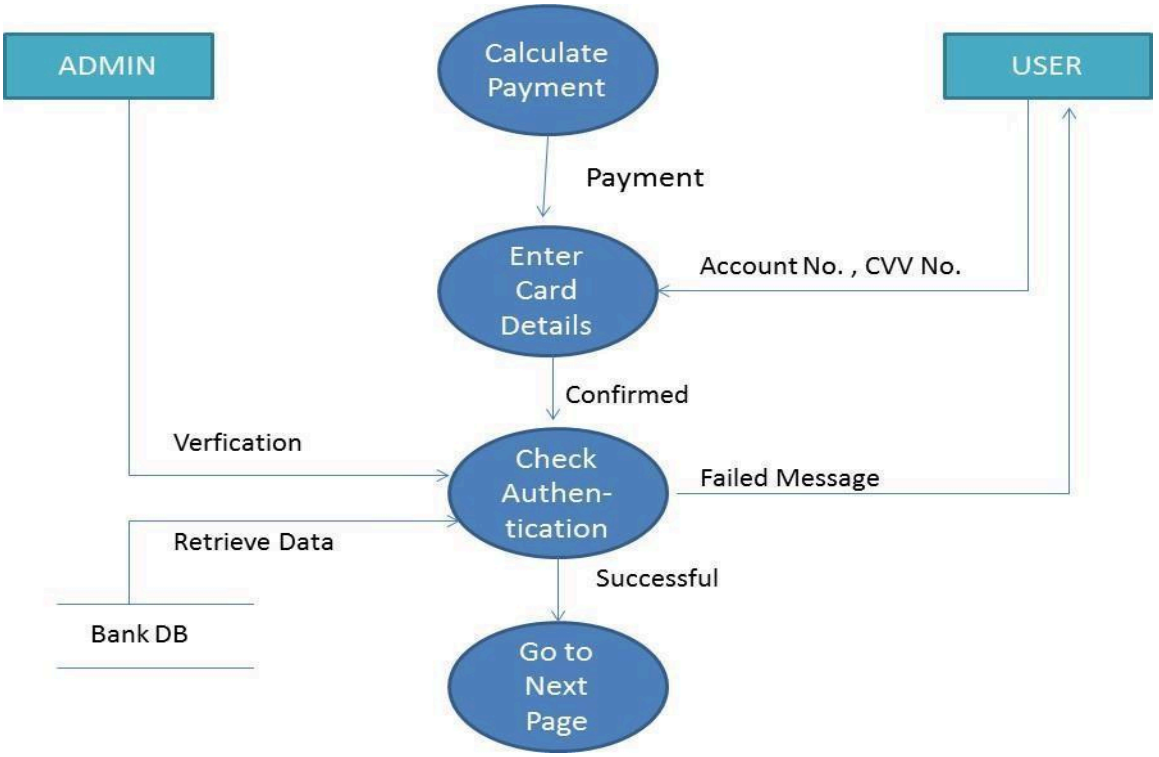
Book Movie Ticket Module:



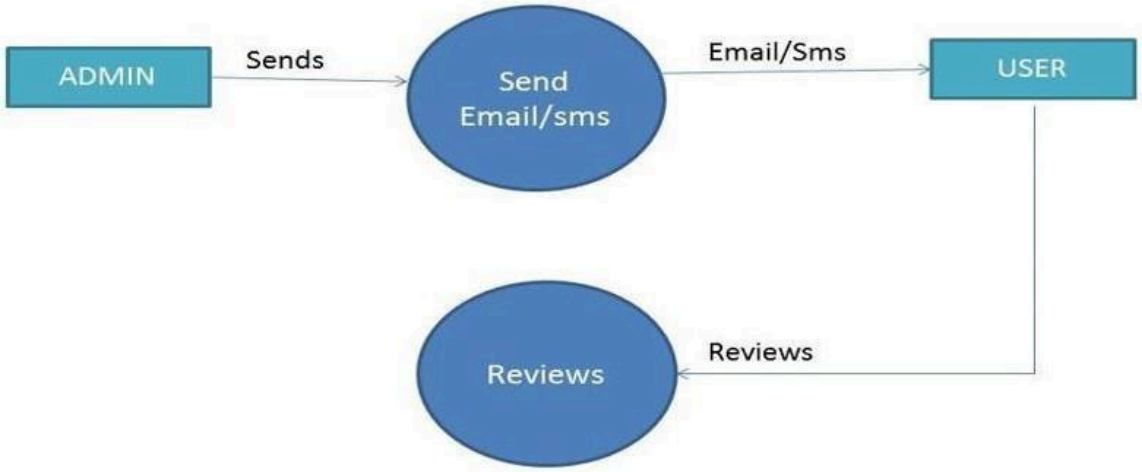
Order Snack Module:



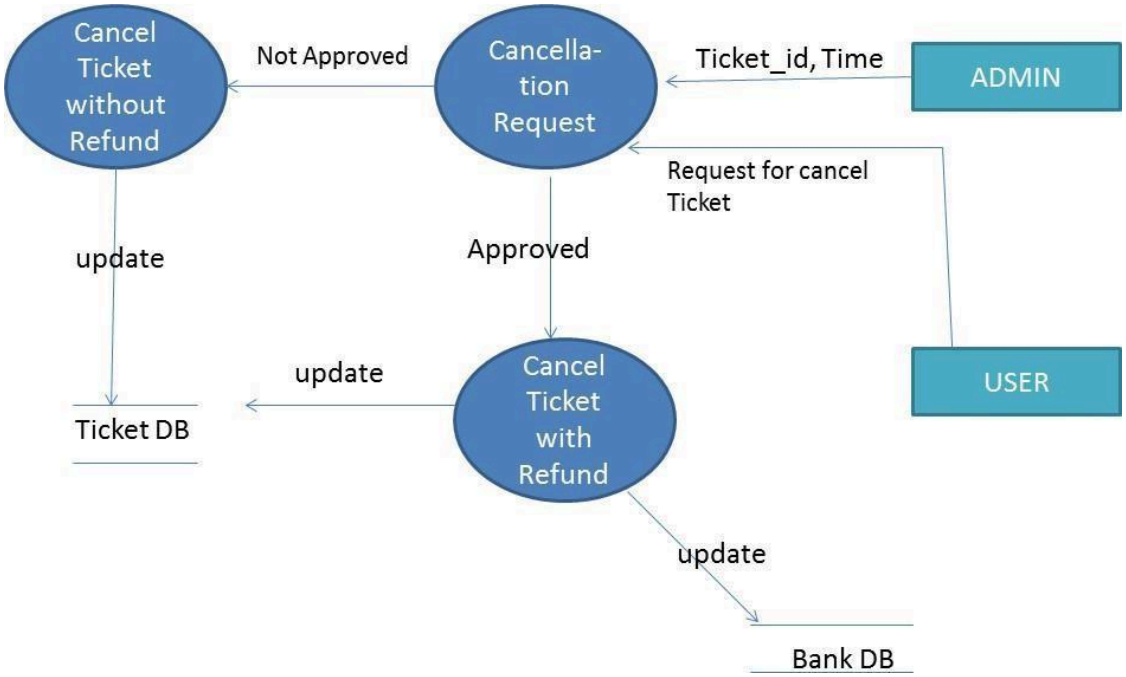
Payment Module:



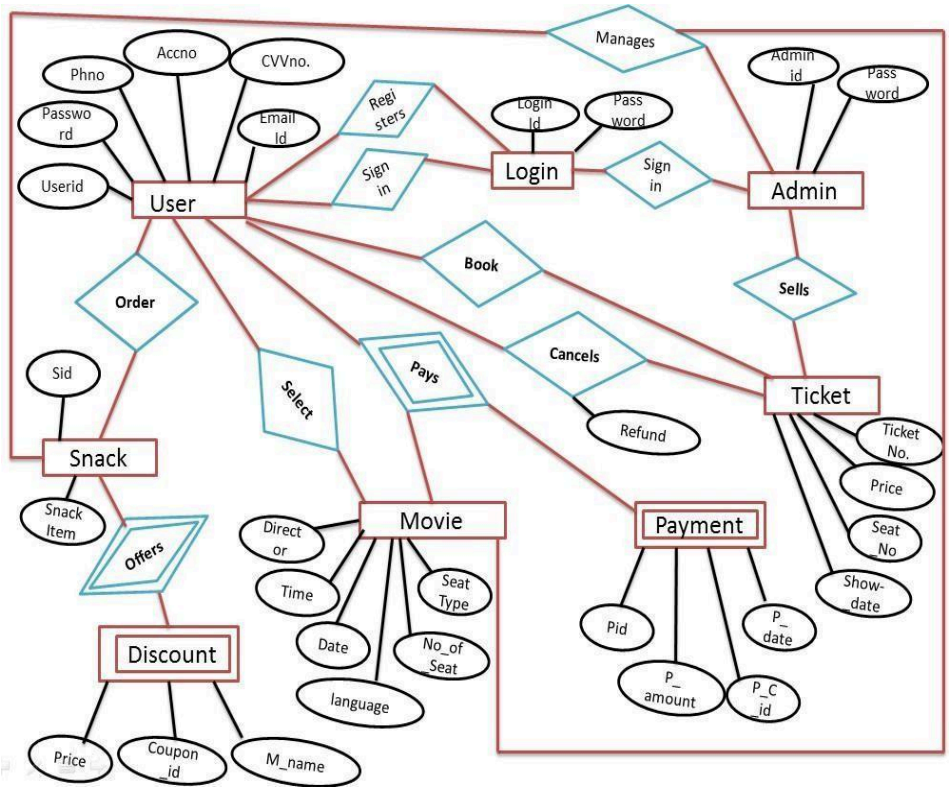
Booking Confirmation Module



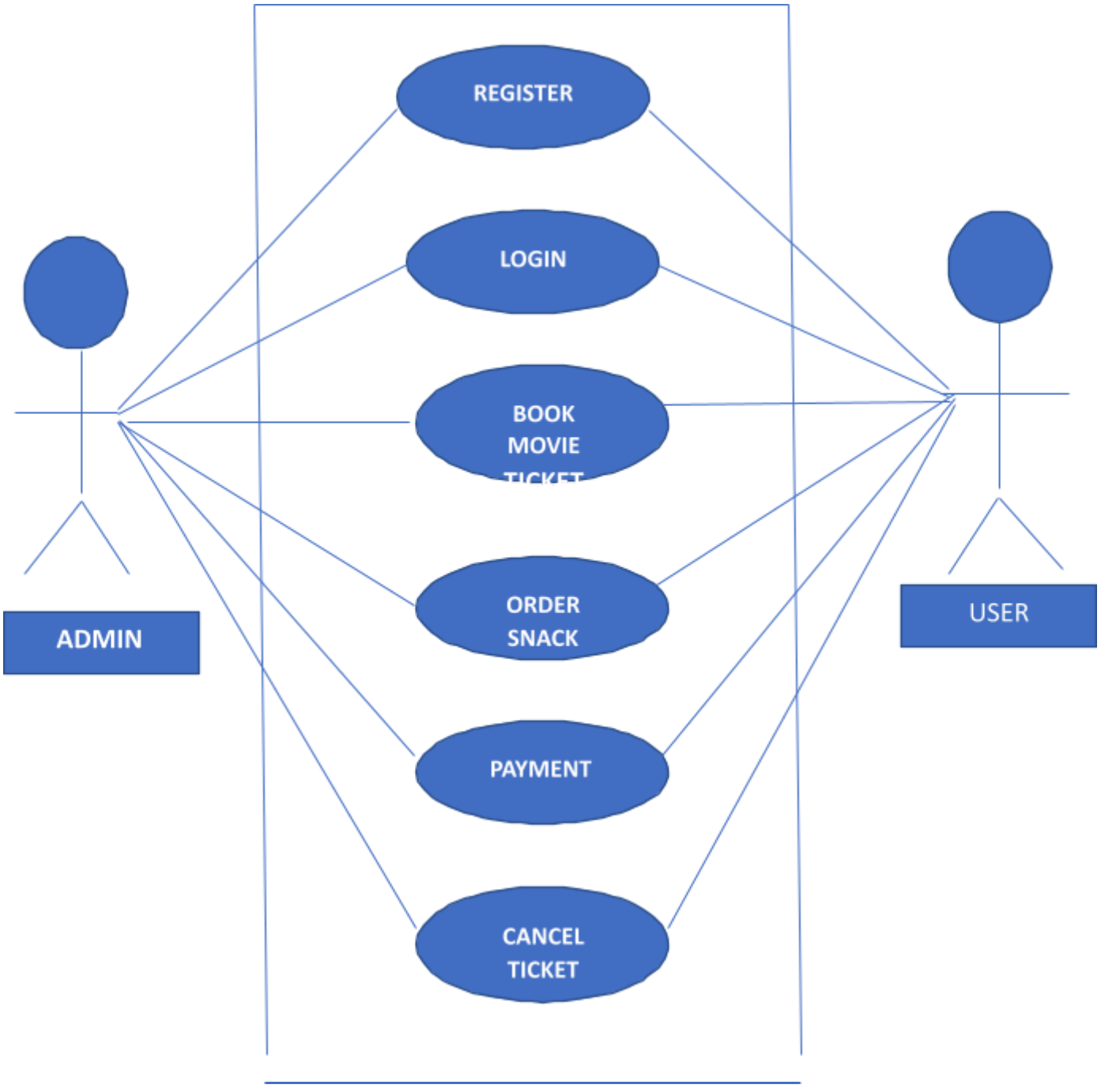
Cancel Ticket Module:



ER DIAGRAM



Data Flow Diagram:



5.3 Flowchart Description

Register

- **Purpose:** It allows users to get registered i.e. to create a new account in the system.
- **Actors involved:** User
- **Precondition:** none
- **Post condition:** User successfully gets registered and the account has been created.
- **Basic Flow:**
    1. System requests the user to login or register.
    2. User clicks on register.
    3. User enters his/her name, password, email id &phone number.
    4. The system validates all the details entered by the user, if correct then it creates a new account of the user.
- **Alternate Flow:** If in the basic flow, the user does not enter any mandatory details or enters any invalid information, the system displays an error message asking the user to re-enter the information. User can either return to Home page or cancel the registering process altogether.
- **Includes:** None
- **Extends:** None

Login

- **Purpose:** It allows the user to use the online movie ticket booking system – with the help of an already existing unique id.
- **Actors involved:** Admin, User
- **Precondition:** Users must have an already existing account in the system. In case of no prior account, user must first register themselves and then log into the system.
- **Post condition:**
    1. On successful login, actor gets logged into the system.

2. If the actor has a role of an admin then he/she would have access to accept user's request and make required updation in the database register, login, change password, payment and cancel ticket.
3. If the actor has a role of user then he/she would have access to register, login, change password, book movie ticket, order snack, payment and cancel ticket.

- **Basic Flow:** 1. User or admin enters his/her id and password.
  2. Then their id and password will be checked for authentication.
- **Alternate Flow:** If the user enters an invalid id and password then the error message will be displayed asking the user to re-enter the details or cancel the login process.
  If the user selects the latter option, the use case ends.
- **Extends:** None
- **Includes:** None

Book Movie Ticket

- **Purpose:** It allows user to book movie ticket online.
- **Actors involved:** User
- **Precondition:** User has to be successfully logged into the system.
- **Post condition:** Movie is booked for the user and all the booking details are displayed to him/her.
- **Basic Flow:** User selects the movie name, date, time, language, type of seat and number of tickets.
- **Alternate Flow:** if the user selects date or time or type of seat that is already booked then error message is displayed to the user and then user can check for that movie on the upcoming dates.
- **Extends:** None
- **Includes:** None

Order Snack

- **Purpose:** It allows the user to order snacks at the time of movie booking.
- **Actors involved:** User

- **Precondition:** User has to first book movie ticket.
- **Post condition:** Snacks has been ordered and user can get it at the time of movie screening in the cinema hall.
- **Basic Flow:** User selects the type of snacks. If there is any coupons/offers user can apply it on the total amount and then can make the payment.
- **Alternate Flow:** If the ordered snack is out of stock then error message is displayed to the user and user can select other item.
- **Excludes:** None
- **Includes:** None

Payment

- **Purpose:** Allows the user to pay the movie ticket bill and snacks bill.
- **Actors involved:** Admin, User
- **Precondition:** User has booked movie ticket and ordered some snacks.
- **Post condition:** After the bill is paid online then the user gets movie ticket and snacks payment confirmation on his/her registered mobile number as well as mail is sent on his/her registered email id.
- **Basic Flow:** User pays the total bill generated through credit card or debit card.
- **Alternate Flow:** If the user enters incorrect CVV no. or credit/debit card no. then error message is displayed to the user to re-enter all the details.
- **Includes:** None
- **Excludes:** None

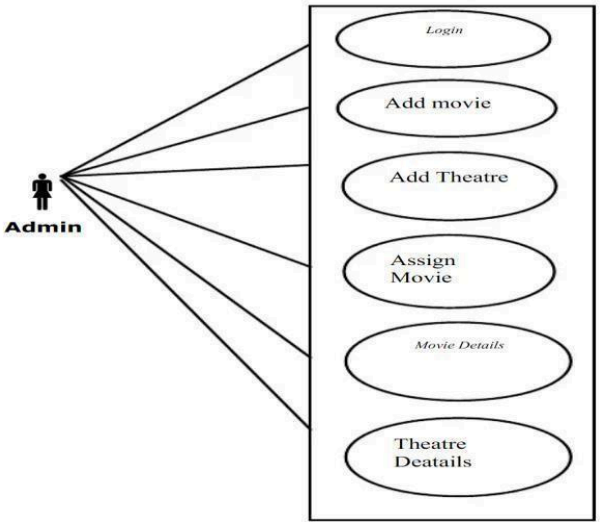
Cancel Ticket

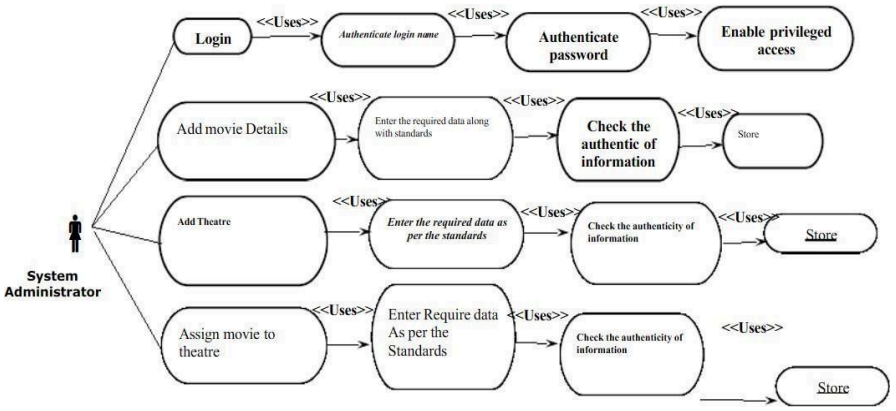
- **Purpose:** Allows user to cancel movie ticket.
- **Precondition:** Ticket should be booked and payment has been made.
- **Post condition:** If the user cancels the movie ticket before 3 hours of the movie time then only 50% refund is being transacted into user's account and the ticket is cancelled otherwise only ticket is cancelled without any refund.

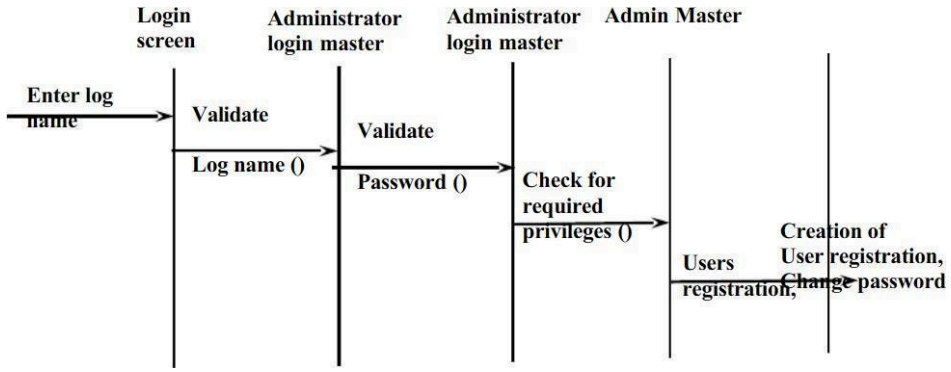
- **Basic Flow:** User raises the request for the movie ticket cancellation by providing details such as date, time and ticket id. Admin checks the details and confirms the request.
- **Alternate Flow**: If the user does not cancel his/her ticket before 3 hours of the movie time then the ticket will not get cancel.
- **Excludes:** None
- **Includes:** None

5.4 Use Case Diagrams

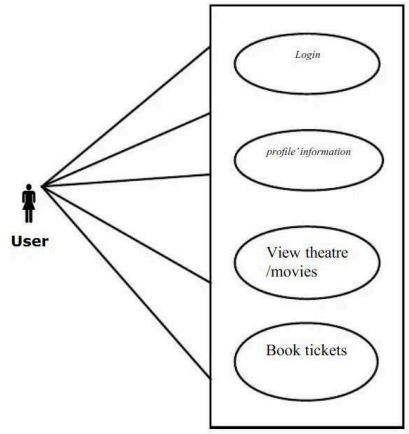
Administrator

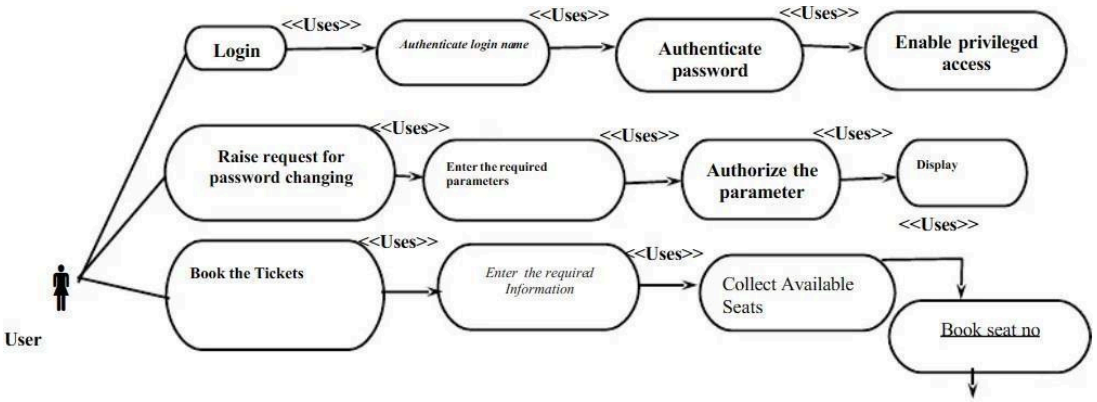


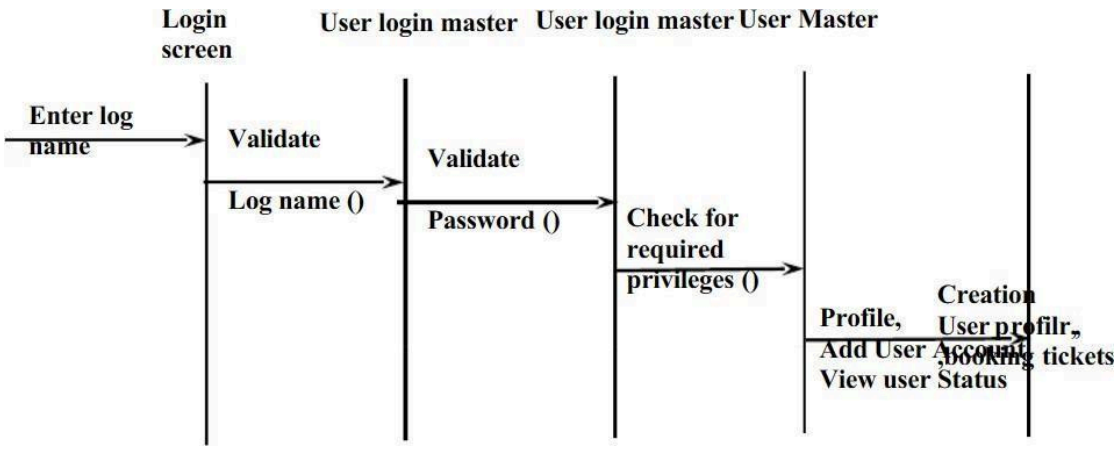




User


Login
profile'information
User
View theatre /movies
Book tickets


<<Uses>>
Login
Authenticate login name
Authenticate password
Enable privileged access
Raise request for password changing
Enter the required parameters
Authorize the parameter
Display
Book the Tickets
Enter the required Information
Collect Available Seats
Book seat no
User


Login screen
User login master
User login master
User Master
Enter log name
Validate
Log name ()
Validate
Password ()
Check for required privileges ()
Profile,
Add User Account
View user Status
Creation
User profilr,
booking tickets

6. Event Coding

==========

```
import mysql.connector


class BookTicket:
    def __init__(self):
        # 1. Establish connection to MySQL server
        self.db = mysql.connector.connect(
            host="localhost",
            user="root",          # Change to your MySQL username if different
            password="S@njay2011"   # Change to your MySQL password
        )
        self.cursor = self.db.cursor()


        # 2. Automatically create database and switch to it
        self.cursor.execute("CREATE DATABASE IF NOT EXISTS movie_booking")
        self.cursor.execute("USE movie_booking")


        # 3. Automatically create the table inside MySQL
        self.cursor.execute("""
            CREATE TABLE IF NOT EXISTS bookings (
                row_no INT,
                seat_no INT,
```

```
        name VARCHAR(50),
        gender VARCHAR(10),
        age INT,
        phone VARCHAR(15),
        price INT,
        PRIMARY KEY (row_no, seat_no)
    )
""")
self.db.commit()

# Input dimensions
print("Enter the number of rows:")
self.rows = int(input())
print("Enter the number of seats in each row:")
self.seats = int(input())

self.l = []
self.userlist = []
self.currentincome = 0
self.noticketpurchased = 0

# Initialize seating matrix
for i in range(self.rows):
    l1 = ['S'] * self.seats
```

```
            l2 = [{}] * self.seats
            self.l.append(l1)
            self.userlist.append(l2)

        # 4. Synchronize memory from existing MySQL records
        self.load_from_db()

        self.menu()

    def load_from_db(self):
        self.cursor.execute("SELECT row_no, seat_no, name, gender, age,
phone, price FROM bookings")
        records = self.cursor.fetchall()
        for rec in records:
            r, s, name, gender, age, phone, price = rec
            if r <= self.rows and s <= self.seats:
                self.l[r - 1][s - 1] = 'B'
                self.userlist[r - 1][s - 1] = {
                    'name': name,
                    'gender': gender,
                    'age': age,
                    'phone': phone
                }
                self.currentincome += price
                self.noticketpurchased += 1
```

```
def show_seats(self):
    count = 1
    print("\n\t\t\tSEATS\n")
    for n in range(1, self.seats + 1):
        if n == 1:
            print('\t%d' % (n), end='\t')
        else:
            print(n, end='\t')
    print('\n')
    for i in self.l:
        print(count, end='\t')
        count += 1
        for j in i:
            print(j, end='\t')
        print('\n')

def user(self):
    self.name = input('\nEnter your name:')
    self.gender = input('Enter your gender:')
    self.age = int(input('Enter your age:'))
    self.phone = input('Enter your phone number:')

def available(self, a, b):
```

```
    if a <= self.rows and b <= self.seats:
        return True


def buy(self):
    self.user_row = int(input('\nEnter the row of your seat:'))
    self.user_seat = int(input('Enter the seat number:'))
    if self.available(self.user_row, self.user_seat):
        print('\nRow', self.user_row, ' seat number:', self.user_seat)
        ticket_price = self.priceofticket(self.user_row)
        print("Cost of the seat is $", ticket_price)
        answer = input("\nChoose 'Yes' or 'No' to proceed :")
        if self.l[self.user_row - 1][self.user_seat - 1] == 'S':
            if answer == 'Yes' or answer == 'yes':
                self.user()
                self.l[self.user_row - 1][self.user_seat - 1] = 'B'
                self.userlist[self.user_row - 1][self.user_seat - 1] = {
                    'name': self.name,
                    'gender': self.gender,
                    'age': self.age,
                    'phone': self.phone
                }
                self.currentincome += ticket_price
                self.noticketpurchased += 1
```

```
                # 5. Insert directly into MySQL and commit
                sql = """INSERT INTO bookings (row_no, seat_no, name,
gender, age, phone, price)
                        VALUES (%s, %s, %s, %s, %s, %s, %s)"""
                val = (self.user_row, self.user_seat, self.name, self.gender,
self.age, self.phone, ticket_price)
                self.cursor.execute(sql, val)
                self.db.commit()


                print("Your Ticket is booked")
            else:
                return
        else:
            print("The seat is already booked")
    else:
        print("\n!!!!!!Seat doesn't exist!!!!!!")


def priceofticket(self, a):
    if self.rows * self.seats <= 60:
        self.price = 10
    else:
        if a <= (self.rows // 2):
            self.price = 10
        else:
            self.price = 8
```

```
        return self.price


    def seat_info(self):
        a = int(input("\nEnter the row no.:"))
        b = int(input("Enter the seat no.:"))
        if self.available(a, b):
            # 6. Query record directly from MySQL table
            self.cursor.execute("SELECT name, gender, age, phone FROM
bookings WHERE row_no = %s AND seat_no = %s", (a, b))
            record = self.cursor.fetchone()
            if record:
                print('Name:', record[0])
                print('Gender:', record[1])
                print('Age:', record[2])
                print('Phone no.:', record[3])
            else:
                print("The seat is empty/available")
        else:
            print("\n!!!!!!Seat doesn't exist!!!!!!")


    def statistics(self):
        self.totalincome = 0
        for i in range(1, self.rows + 1):
            self.totalincome += self.priceofticket(i) * self.seats
        percentage = (self.currentincome / self.totalincome) * 100
```

```
        print("\nNo. of tickets purchased", self.noticketpurchased)
        print("percentage", percentage)
        print("current income: $", self.currentincome)
        print('Total income: $', self.totalincome)


    def menu(self):
        print('\n1.Show the seats\n2.Buy a ticket\n3.Statistics\n4.Show booked ticket user info\n0.Exit')
        choice = int(input("Enter the choice:"))
        if choice == 1:
            self.show_seats()
            self.menu()
        elif choice == 2:
            self.buy()
            self.menu()
        elif choice == 3:
            self.statistics()
            self.menu()
        elif choice == 4:
            self.seat_info()
            self.menu()
        elif choice == 0:
            self.cursor.close()
            self.db.close()
            return None
```

```
movie1 = BookTicket()
```

7. Output

CS PROJECT MOVIE TICKET BOOKING SYSTEM STD.12.py - C:\Users\91981\Desktop\12th\PRACTICAL FILE CS 12TH\CS PROJECT MOVIE TICKET BOOKING SYSTEM STD.12.py (3.11.1)

File Edit Format Run Options Window Help

```
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec  6 2022, 19:58:39) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

= RESTART: C:\Users\91981\Desktop\12th\PRACTICAL FILE CS 12TH\MOVIE TICKET BOOKING SYSTEM 3.py
Enter the number of rows:
9
Enter the number of seats in each row:
19

1.Show the seats
2.Buy a ticket
3.Statistics
4.Show booked ticket user info
0.Exit
Enter the choice:1

                    SEATS

      1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16 17 18 19

1     S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S

2     S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S

3     S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S

4     S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S

5     S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S

6     S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S

7     S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S

8     S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S

9     S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S


1.Show the seats
2.Buy a ticket
3.Statistics
4.Show booked ticket user info
0.Exit
```

```
Enter the number of rows:
9
Enter the number of seats in each row:
19

1.Show the seats
2.Buy a ticket
3.Statistics
4.Show booked ticket user info
0.Exit
Enter the choice: 1
```

```
                    SEATS
      1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16 17 18 19
1     S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S
2     S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S
3     S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S
...
9     S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S  S
```

CS PROJECT MOVIE TICKET BOOKING SYSTEM STD.12.py - C:\Users\91981\Desktop\12th\PRACTICAL FILE CS 12TH\CS PROJECT MOVIE TICKET BOOKING SYSTEM STD.12.py (3.11.1)

File Edit Format Run Options Window Help

```
Enter the choice:2

Enter the row of your seat:8
Enter the seat number:4

Row 8  seat number: 4
Cost of the seat is $ 8

Choose 'Yes' or 'No' to proceed :yes

Enter your name:Nandini Narayanan
Enter your gender:Female
Enter your age:17
Enter your phone number:9876543210
Your Ticket is booked

1.Show the seats
2.Buy a ticket
3.Statistics
4.Show booked ticket user info
0.Exit
Enter the choice:2

Enter the row of your seat:8
Enter the seat number:5

Row 8  seat number: 5
Cost of the seat is $ 8

Choose 'Yes' or 'No' to proceed :yes

Enter your name:Archita Gupta
Enter your gender:Female
Enter your age:17
Enter your phone number:975318642
Your Ticket is booked

1.Show the seats
2.Buy a ticket
3.Statistics
4.Show booked ticket user info
0.Exit
Enter the choice:2

Enter the row of your seat:8
Enter the seat number:3
```

CS PROJECT MOVIE TICKET BOOKING SYSTEM STD.12.py - C:\Users\91981\Desktop\12th\PRACTICAL FILE CS 12TH\CS PROJECT MOVIE TICKET BOOKING SYSTEM STD.12.py (3.11.1)

File Edit Format Run Options Window Help

```
Row 8  seat number: 3
Cost of the seat is $ 8

Choose 'Yes' or 'No' to proceed :yes

Enter your name:Sanika Karwa
Enter your gender:Female
Enter your age:17
Enter your phone number:1262157289
Your Ticket is booked

1.Show the seats
2.Buy a ticket
3.Statistics
4.Show booked ticket user info
0.Exit
Enter the choice:2

Enter the row of your seat:8
Enter the seat number:5

Row 8  seat number: 5
Cost of the seat is $ 8

Choose 'Yes' or 'No' to proceed :no
The seat is already booked

1.Show the seats
2.Buy a ticket
3.Statistics
4.Show booked ticket user info
0.Exit
Enter the choice:2

Enter the row of your seat:8
Enter the seat number:3

Row 8  seat number: 3
Cost of the seat is $ 8

Choose 'Yes' or 'No' to proceed :yes
The seat is already booked

1.Show the seats
2.Buy a ticket
3.Statistics
```

CS PROJECT MOVIE TICKET BOOKING SYSTEM STD.12.py - C:\Users\91981\Desktop\12th\PRACTICAL FILE CS 12TH\CS PROJECT MOVIE TICKET BOOKING SYSTEM STD.12.py (3.11.1)

File Edit Format Run Options Window Help

```
1.Show the seats
2.Buy a ticket
3.Statistics
4.Show booked ticket user info
0.Exit
Enter the choice:3

No. of tickets purchased 3
percentage 1.5789473684210527
current income: $ 24
Total income: $ 1520

1.Show the seats
2.Buy a ticket
3.Statistics
4.Show booked ticket user info
0.Exit
Enter the choice:4

Enter the row no.:8
Enter the seat no.:3
Name: Sanika Karwa
Gender: Female
Age: 17
Phone no.: 1262157289

1.Show the seats
2.Buy a ticket
3.Statistics
4.Show booked ticket user info
0.Exit
Enter the choice:4

Enter the row no.:8
Enter the seat no.:4
Name: Nandini Narayanan
Gender: Female
Age: 17
Phone no.: 9876543210

1.Show the seats
2.Buy a ticket
3.Statistics
4.Show booked ticket user info
0.Exit
Enter the choice:4
```

Ln: 44 Col: 0

```
Enter the row no.:8
Enter the seat no.:5
Name: Archita Gupta
Gender: Female
Age: 17
Phone no.: 975318642

1.Show the seats
2.Buy a ticket
3.Statistics
4.Show booked ticket user info
0.Exit
Enter the choice:0
```

One of the key features of the **Movie Ticket Booking System** is the ability to display details of a booked ticket. This ensures complete **transparency and reliability** in the booking process.

When a user chooses option **4. Show booked ticket user info**, the system asks for the row and seat number. Based on the stored records, it displays the personal details of the customer who booked that particular seat.

For example

- **Row No.: 8**
- **Seat No.: 5**
- **Name:** Archita Gupta
- **Gender:** Female
- **Age:** 17
- **Phone No:** 97531864

8.Conclusion

The Movie Ticket Booking System project successfully demonstrates how technology can streamline and simplify the ticket booking process for both customers and cinema administrators. By integrating features such as movie listings, real-time seat availability, user authentication, and secure payment options, the system enhances user convenience and operational efficiency.
This project has provided valuable insights into the development of a full-stack application, incorporating both front-end and back-end technologies. It emphasizes the importance of user-friendly interfaces, data validation, and secure transactions in real-world applications.

Overall, the Movie Ticket Booking System not only meets the current needs of moviegoers but also lays a strong foundation for future enhancements like mobile integration, AI-based recommendations, and multi-language support. It stands as a practical and scalable solution in the growing field of digital entertainment services.

9.Bibliography

1. Website: https://www.youtube.com
2. Website: https://www.w3resource.com
3. Website: www.wikipedia.org
4. Website: www.google.com/Python project